CredOne RCM - UMS Architecture

Enterprise User Management & Role-Based Access Control Blueprint

Derived from Production-Proven Employer Dashboard Architecture | Reference Implementation

TARGET MODULE:	CredOne RCM User Management System (UMS) & Access Governance
TECH STACK:	Next.js 16 (App Router), React 19, MongoDB, SWR, Joi, Cloudinary, Tailwind
PREPARED FOR:	Credence Engineering Team / Technical Leadership Review

1. Executive Architectural Blueprint

The User Management System (UMS) is designed as a zero-trust, hierarchical access control backbone. It decouples business domain actions (claims, billing, reports) from authorization rules while providing seamless user lifecycle transitions: self-service provisioning, multi-attribute access control (ABAC/RBAC), automated organizational hierarchy delegation, and audit-compliant soft offboarding.

System Tier	Technology	Core Responsibilities & Implementation Details
Client & Presentation	React 19 / Next.js 16 CSR/SSR	Role-routed dashboards (/admin, /manager, /employee), Real-time SWR cache, Framer Motion transitions, responsive sidebar navigation.
State & Session Layer	RootContext + LocalStorage	Centralized authentication state, token storage, reactive toast dispatcher, role routing guards, and session hydration.
API & Security Gate	Next.js App Router API Routes	Edge & Node runtime handlers (/api/employees, /api/ar, /api/leave, /api/eod). Joi schema validation, multipart payload decoding.
Data Engine & Storage	MongoDB Atlas + Cloudinary	Collection connection pooling via clientPromise, subdocument arrays for attendance/leave matrices, secure Cloudinary doc vault.

CORE ENGINEERING HIGHLIGHTS:

1. Active Status Gatekeeper: Inactive accounts are blocked immediately at the auth root, eliminating orphan sessions.
2. Dynamic Code Generation: Auto-generates sequential employee/agent codes with department prefixes (e.g. SEZT001, SEZS001).
3. Fail-Safe Approval Fallback: If a reporting manager is on leave or absent, approvals escalate automatically to HR Admins.
4. Non-Destructive Soft Delete: Status toggle preserves historical financial, payroll, and audit trails required for compliance.

2. Complete User Lifecycle Execution Flow

Phase	Trigger & Actor	Backend Operations & Database Mutations	Security / Outcome
1. Provisioning	Admin submits AddEmployeeForm	POST /api/employees. Validates Joi payload. Calculates max sequence number in MongoDB. Generates code (e.g. SEZT042). Sets isActive: true.	Account created, default attendance matrix initialized, credentials ready.
2. Authentication	User submits credentials on /login	Queries employee by email. Validates password hash/match. Checks isActive === true. If false, rejects with 403 / Account Disabled.	Returns authenticated user context with role & reporting metadata.
3. Dynamic Routing	Client LoginRoot handler	Reads role from response: super_admin / admin -> /dashboard/admin; manager -> /dashboard/manager; employee -> /dashboard/employee.	Isolated portal boundary. User cannot access unauthorized views.
4. Hierarchy Delegation	Employee applies for Leave / AR	PUT /api/leave or /api/ar. Attaches employeeId, reportingManagerId. Updates manager badge queue in real-time via SWR mutation.	Targeted approval routing. Absence check falls back to Admin.
5. Soft Offboarding	Admin toggles active toggle	DELETE /api/employees?id=XYZ or PUT isActive: false. Sets modified_on timestamp. Flags user account as inactive.	Session revoked instantly; all past attendance, EOD, payroll preserved.

3. Backend Runtime Architecture & MongoDB Driver

The backend is built natively on Next.js 16 Route Handlers (Edge & Node.js runtimes). Instead of re-opening MongoDB client connections on each HTTP request, it utilizes a global singleton clientPromise to pool database connections across hot serverless lambdas. This guarantees high throughput and sub-50ms query response times.

Backend Component	Implementation File	Key Architecture Details
Connection Pooler	*@/app/components/lib/db.js*	Singleton MongoClient caching via global._mongoClientPromise in development; reusable persistent pool in production.
Schema Validation	*@/app/api/employees/route.js*	Joi validation schema enforcing email syntax, mobile regex (/^[0-9]{10,15}$/), roles (employee, manager, admin), and date constraints.
Dual-Mode PUT Handler	*@/app/api/employees/route.js*	Inspects Content-Type: multipart/form-data processes Cloudinary binary document streams; application/json performs granular field updates.
Bi-directional Sync	*@/app/api/employees/route.js*	When a manager or admin approves a leave in employee subdocuments, the engine auto-updates corresponding dates in attendance[month].status.

4. RESTful API Specification & Parameters

Method & Endpoint	Headers & Payload	Query Parameters	Operation Description
GET /api/employees	Authorization: Bearer Session	?id=<ObjectId>&email=<str>&name=<str>&includeInactive=true	Fetches single employee by ID/email or searches by name regex. Filters out inactive unless explicitly requested.
POST /api/employees	Content-Type: application/json (Joi validated body)	None	Creates employee. Runs generateNextCode() to calculate next sequence. Sets isActive=true, defaults attendance to {}.
PUT /api/employees	application/json OR multipart/form-data	None	Updates profile data, bank accounts, or toggles isActive. Handles Cloudinary binary uploads for experience/degrees.
DELETE /api/employees	Authorization: Admin Bearer	?id=<ObjectId>&type=experience&index=0	If type omitted: Executes soft-deactivate (isActive=false). If type specified: removes nested document entry.
PUT /api/leave/approve	application/json { adminId, employeeId, startDate, endDate, status }	None	Approves or rejects pending leave. Automatically propagates "approved" status into employee attendance matrix.
PUT /api/ar	application/json { requestId, action, remarks, adminId }	None	Regularizes missed check-in/out stamps. Mutates attendance hours and shifts.
GET /api/eod	Authorization: Session	?role=employee&id=<id>&date=YYYY-MM-DD	Fetches daily morning task plan and evening EOD submission status for productivity tracking.

5. MongoDB Collections & Entity Schema

Field Name	BSON Type	Validation Rules	Architectural Purpose
_id	ObjectId	Auto-generated primary key	Unique system identifier for references.
employeeCode	String	Regex: ^[A-Z]{3,4}[0-9]{3}$	Human-readable unique identifier (e.g. SEZT008).
name, email, mobile	String	Required, Email syntax, 10-15 digits	Primary user identity & login credentials.
role	String	Enum: [employee, manager, admin, super_admin]	Defines RBAC permission matrix & routing.
reportingId	String / ObjectId	Valid 24-char ObjectId / null	Points to Manager _id for hierarchical approvals.
isActive	Boolean	Default: true (Required for login)	Controls access instantly without losing audit history.
attendance	Object (Map)	Keyed by MonthName -> Array of Day Records	Stores clock-in, clock-out, work hours, leave statuses.
leaves	Array of Subdocs	{ startDate, endDate, reason, status: pending|approved|rejected }	Leave audit history with approvals and manager IDs.

6. Applying This Blueprint to CredOne RCM UMS

Employer Dashboard Pattern	CredOne RCM UMS Implementation	HIPAA / Revenue Cycle Advantage
Role Hierarchy (SuperAdmin -> Admin -> Manager -> Employee)	Super Admin -> Billing Director -> QA Lead -> AR Caller / Medical Coder	Enforces Principle of Least Privilege: Coders only view Assigned Queues; only Managers can approve write-offs.
Reporting Hierarchy (reportingId)	Agent to Lead & Payer/Provider Mapping	Prevents claim collision; ensures high-dollar claims (> $1,000) are assigned to experienced senior specialists.
isActive Gatekeeper & Soft Offboard	Immediate HIPAA Revocation & Audit Preservation	Instant termination of PHI access upon resignation while preserving claim submission trails for federal audits.
Absence Fallback Approval Engine	Escalation for High-Priority Denial Appeals	If an assigned Team Lead is out of office, pending claim write-offs escalate to Billing Manager to prevent timely filing denials.

7. Ready-to-Use Speaking Script for Your Manager Discussion

"When I designed the Employer Dashboard, I focused on three non-negotiable architectural principles:"

1. Clean Separation of Identity and Permissions: We implemented an RBAC layer with dynamic code generation and an active status gatekeeper. Inactive employees cannot pass the login root, securing company resources.

2. Resilient Approval Hierarchies: Team members report to managers via a structured reportingId. To prevent business bottlenecks, we built a presence-aware fallback engine: if a manager is absent, requests auto-escalate to Admins.

3. Audit-Grade Data Handling: Rather than hard-deleting records, we built soft-deactivation. This protects historical attendance, payroll, and activity trails from being orphaned.

"For CredOne RCM, we can apply this exact blueprint to support medical billers, coders, and team leads, ensuring strict HIPAA compliance, granular claim queues, and audit-ready access logs."